

Altitude Control of Tello Mini Drone Using Visual Feedback

1. Introduction

1.1 The Ryze Tello Mini Drone

The Ryze Tello Drone is a compact lightweight quadrotor drone developed by Ryze Tech in Collaboration with DJI and Intel. Designed for research and educational application it weighs approximately 80g and measures 98 x 92.5 x 41 mm. Despite its small form factor, Tello is equipped with a vision positioning system (VPS) for indoor stability, a forward-facing camera and an Electronic Speed Controller (ESC) managing each of its four brushless motors.

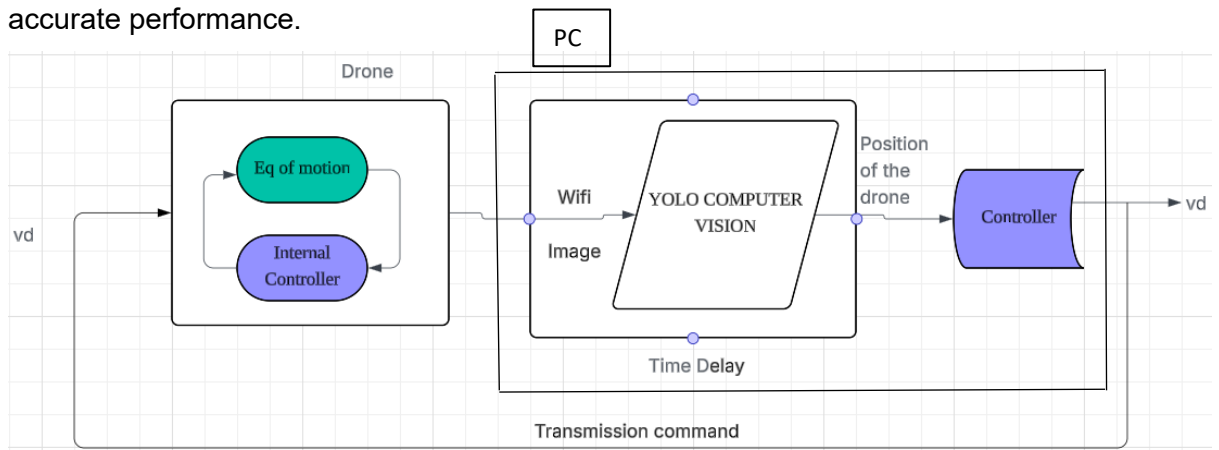
- Weight: approx. 80 g (including propeller and battery)
- Size: 98 x 92.5 x 41 mm
- Propeller: 3 inches
- Built-in features: rangefinder, barometer, LED, vision system, 2.4 GHz 802.11n Wi-Fi, 720p live view, Camera
- Ports: Micro USB charging port

Communication with the drone is established over Wi-Fi using Tello's UDP-based SDK. Commands are sent as ASCII strings (e.g., rc roll pitch throttle yaw), and telemetry including height is streamed back at approximately 10 Hz. In this project the rc_value (throttle velocity command) was used as the control input u , and the measured altitude in centimetres was used as the system output y .

1.2 System Configuration

- **Physical Layout and Reference System:** The experiment setup is designed around a simple and fixed reference system to track the altitude of the drone accurately. A tennis ball is placed at a distance of 0.9 meters in front of the drone's front camera. This tennis ball acts as a visual reference point, helping the system understand the drone's vertical position using camera input.

- System Configuration and feedback loop:** The system follows an off-board control architecture, where most of the processing is performed on an external computer rather than on the drone itself. The drone continuously sends live video stream is processed in real time using the YOLO (You Only Look Once) object detection algorithm, which detects the tennis and extracts its vertical position in terms of pixel This position is then compared with the predefined reference value of 200 pixels. The difference between the current position and the reference position is treated as an error. Based on this error, a controller calculates the required vertical velocity command vd , which is sent back to the drone to adjust its height accordingly. In this way, a closed-loop feedback system is formed, where the drone continuously corrects its position based on visual input. However, since the system relies on wireless communication and external processing, there is a small-time delay involved in data transmission and computation. This delay is an important factor and must be considered during system modelling and controller design to ensure stable and accurate performance.



- Mathematical Derivation of the Internal Model**
- Dynamics and Control Law**

The vertical motion is governed by Newton's Second Law. We assume the drone has an internal controller that compensates for gravity and attempts to match a reference velocity.

- Equation of Motion:**

$$m\ddot{y} = f - mg \quad (1)$$

- Internal PD-like Control Law:**

The force f is generated based on the error between the drone's actual velocity (v) and the commanded velocity (vd):

$$f = -k_p(v - v_d) - k_d(\dot{v} - \dot{v}_d) + mg \quad (2)$$

$$m\ddot{y} = -k_p(\dot{y} - v_d) - k_d(\ddot{y} - \dot{v}_d)$$

$$(m + k_d)\ddot{y} + k_p\dot{y} = k_p v_d + k_d \dot{v}_d$$

- **Laplace Transform :**

$$s^2(m + k_d)Y(s) + sk_p Y(s) = (k_p + sk_d)V_d(s)$$

$$Y(s)[s^2(m + k_d) + sk_p] = V_d(s)[k_d s + k_p]$$

$$G(s) = \frac{Y(s)}{V_d(s)} = \frac{k_d s + k_p}{(m + k_d)s^2 + k_p s} \quad (3)$$

- **Transfer Function:**

$$G(s) = \frac{\frac{k_d s + k_p}{m + k_d}}{s^2 + \frac{k_p}{m + k_d} s} \quad (4)$$

To make our model realistic for the Tello drone environment, we must account for the dead time (d) caused by Wi-Fi transmission and YOLO processing.

- **The Time-Delayed Differential Equation:** In the time domain, the command (u) does not affect the drone instantly. Instead, the drone reacts to a command sent (d) seconds ago. We rewrite our previous differential equation:

$$\ddot{y}(t) + \frac{k_p}{m + k_d}\dot{y}(t) = \frac{k_p}{m + k_d}u(t - d) + \frac{k_d}{m + k_d}\dot{u}(t - d)$$

$$P(s) = \frac{\frac{k_d s + k_p}{m + k_d}}{s^2 + \frac{k_p}{m + k_d} s} e^{-sd} \quad (5)$$

1.3 Project Objective

The goal of this project is to design and validate an altitude controller for the Tello drone. Altitude control is a fundamental requirement for any autonomous aerial system without a well-tuned vertical controller the drone cannot hover stably, track a target, or execute precise manoeuvres.

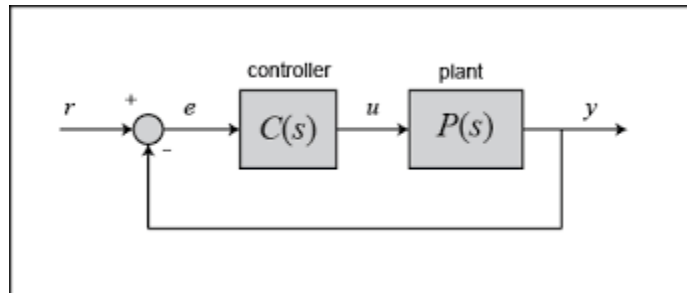


Fig 1.1 : closed-loop feedback control system

The figure (1.1) illustrates a basic closed-loop feedback control system. The reference input (r) is compared with the output (y) at the summing junction to generate the error signal (e). This error is processed by the controller ($C(s)$), which produces the control input (u) applied to the plant ($P(s)$). The plant generates the system output (y), and this output is fed back to the input for continuous comparison with the reference. Such feedback architecture is widely used in control engineering because it improves tracking performance, reduces error, and enhances system robustness.

2. Data Collection & System Identification

2.1 Experimental Data Collection

Before any model could be built, real flight data had to be collected. The drone was flown in a controlled indoor environment and two experiments were performed:

- **Step Response Test:** A constant throttle command was applied and the drone's altitude response over time was logged. This reveals key dynamic properties such as rise time, settling time.
- **Non-Periodic (Pseudo-Random) Input Test:** Varying throttle commands were applied to excite a broad range of system frequencies. This type of experiment is essential for robust system identification as it prevents the model from overfitting to a single operating point.

In this experiment, the drone was connected to the PC through Wi-Fi and maintained at a distance of 0.9 m from the target ball. The input to the system was the velocity command, while the output was the altitude y_1 , which was obtained by tracking the ball using YOLO-based computer vision through the PC camera. Using the developed code, the drone was controlled accordingly, and the corresponding response data were recorded and saved in a CSV file. The step response and the non-periodic dataset were then plotted, and the time delay was calculated from the graph, where the x-axis represented time and the y-axis represented altitude y_1 .

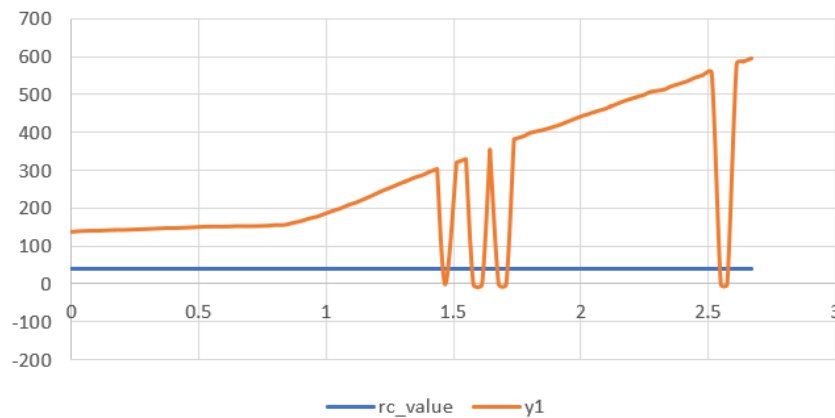


Fig 2.1 (Step Response Result)

The (Fig 2.1) graph shows the relationship between the reference command *rc value* and the measured output y_1 over time. The blue line represents the reference signal, which remains nearly constant, while the orange line shows the altitude response of the drone obtained from ball tracking. The output generally increases as the system responds, but there are a few sharp drops to zero; these correspond to moments when the ball was not visible to the camera, so the measured value was recorded as zero. The time delay observed from the graph was 0.803 s, indicating the delay between the input command and the corresponding system response

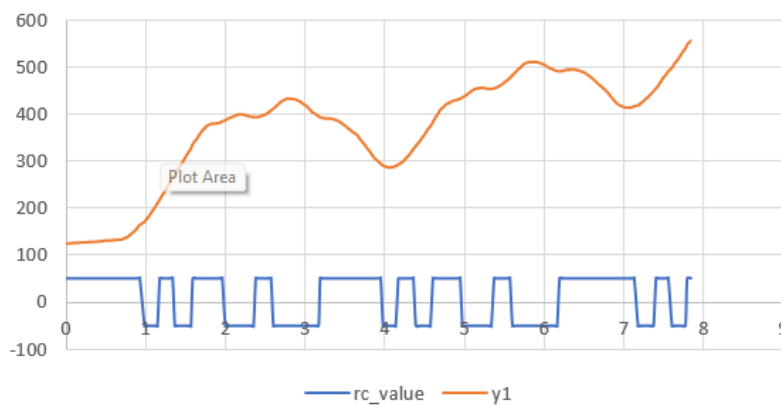


Fig 2.2 (Non-Periodic Result)

The (Fig 2.2) graph represents the response of the drone for a non-periodic dataset, where the reference control value *rc value* changes at different intervals while the drone tracks the ball using computer vision. The blue curve shows the varying input velocity command, and the orange curve shows the corresponding output altitude *y1*. As the reference value changes, the measured altitude follows with a noticeable delay, showing the dynamic response of the system. The graph also shows that the output does not remain constant but changes continuously according to the tracking behavior and the motion of the ball. The measured time delay for this dataset was 0.67 s, which indicates the lag between the input change and the observed output response.

2.3 MATLAB System Identification

The recorded CSV data was loaded into MATLAB and formatted as an *iddata* object the standard input-output data structure (Property of the Drone) required by the System Identification Toolbox:

```
data=readtable("yolo_log_ver5_20260325.csv");

u=data.rc_value;          % Control input (throttle velocity)
y=data.y1;                % System output (altitude in cm)

ts=mean(diff(data.time)); % Average sampling time
myDroneD=iddata(y,u,ts); % Format as iddata object

systemIdentification;    % Open System Identification App
```

Inside the System Identification App, the *iddata* object was imported, and a transfer function model was fitted to the experimental data obtained from the drone altitude tracking experiments. The app Used the Levenberg-Marquardt optimisation algorithm, a robust nonlinear least-squares method widely used for system identification due to its ability to blend the stability of gradient descent with the fast convergence of the Newton-Raphson method. This algorithm iteratively adjusted the numerator (1 zero) and denominator (2 poles) coefficients of the transfer function until the model output best matched the measured drone altitude. The identified model achieved a fit of 87.37%, demonstrating that the transfer function effectively captures the essential dynamics of the system with good accuracy.

2.4 Identified Transfer Function

The System Identification App returned the following transfer function from input *u1* (*rc_value*) to output *y1* (altitude):

```

From input "u1" to output "y1":
              0.2335 s + 23.53
exp(-0.67*s) * -----
              s^2 + 3.721 s + 1.972e-10

```

- **$e^{-0.67s}$** : Pure transport delay of 0.67 s identified from the Excel time-series plot.
- **Numerator (0.2335s + 23.53)**: A first-order numerator indicating the system has a finite zero; the large constant (23.53) implies significant steady-state sensitivity to the input.
- **Denominator ($s^2 + 3.7721s + 1.972 \times 10^{-10}$)**: A second-order denominator. The near-zero constant term ($\sim 10^{-10}$) in the denominator effectively introduces an integrator into the system, meaning the drone's altitude will accumulate indefinitely without feedback consistent with physical intuition (a drone with constant thrust will keep rising).
- **From Identified Transfer Function we get**

$$P(s) = \frac{bs+c}{s^2+as} \quad a = 3.72 \quad b = 0.2335 \quad c = 23.53$$

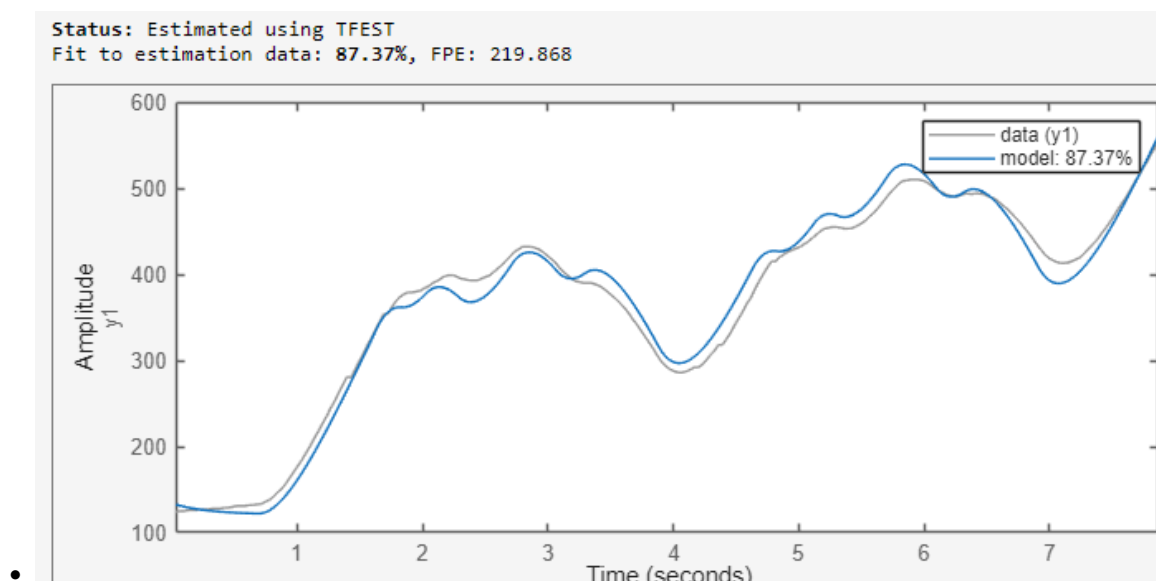


Fig (2.4.1) model validation results

This figure presents the model validation results from the System Identification App, comparing the measured drone altitude data (Gray line) with the fitted transfer function model output (blue line). The model closely tracks the experimental data across the time series, demonstrating its ability to capture the system's dynamics effectively. The achieved model fit of 87.37% indicates high accuracy, with the optimization process requiring 18 iterations and 328 function evaluations using the Levenberg-Marquardt

algorithm. The convergence stopped when no further improvement was possible along the search direction with line search, confirming a robust fit for the drone altitude response.

3.Controller Design

3.1 Pole Placement

In an altitude control system, the position of the closed-loop poles on the s-plane shows how the system reacts to a change in height or to a disturbance like a gust of wind. When all poles are on the left side of the s-plane, any altitude error naturally dies out with time and the drone settles back to the commanded height. If poles lie on the vertical imaginary axis, the altitude keeps oscillating and never fully settles, and if any pole is on the right side, the oscillations grow and the drone becomes unstable and unsafe.

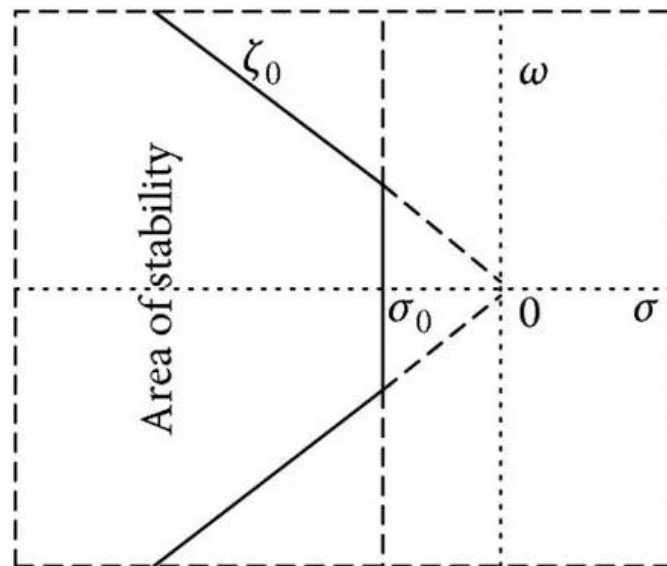


Fig (3.1)

Figure 3.1 shows not just “stable vs unstable,” but a more strict region where the poles should lie to get good performance. The horizontal axis (σ) is the real part of the poles and the vertical axis is the imaginary part (ω). The vertical line at σ_0 sets how fast the response should decay (faster decay is further to the left), and the two slanted lines marked by ζ_0 set a minimum damping level so that overshoot and oscillations are limited. The shaded wedge labeled “area of stability” is where the closed-loop poles should be placed so the altitude controller is not only mathematically stable but also settles quickly and smoothly, without excessive oscillations.

3.2 PD Controller

A PD controller uses the present altitude error and the rate at which this error is changing to shape the response of the altitude loop. The control input is written in the form

$$u(t) = K_p e(t) - K_d \frac{dy(t)}{dt} \quad (6)$$

where $e(t)$ is the altitude error, K_p is the proportional gain, and K_d is the derivative gain. The proportional term pulls the drone back toward the desired altitude, while the derivative term acts like a predictive damping term that looks at how fast the error is changing and reduces overshoot and oscillations. To design the PD controller systematically, the desired closed-loop dynamics are first specified by choosing two target poles λ_1 and λ_2 in the stable region of the s-plane. These poles define a desired second-order characteristic polynomial

$$P(s) = \frac{bs+c}{s^2+as} \quad C(s) = K_p - sK_d$$

$$G(s) = \frac{P(s)C(s)}{1+P(s)C(s)}$$

$$G(s) = \frac{s^2+as+(bs+c)(K_p-sK_d)}{(bs+c)(K_p-sK_d)}$$

$$s^2 + as + bK_p s + cK_p - bK_d s^2 - cK_d s$$

Rearranging:

$$(1 - bK_d)s^2 + (a + bK_p - cK_d)s + cK_p$$

$$(s - \lambda_1)(s - \lambda_2) = 0$$

$$s^2 - (\lambda_1 + \lambda_2)s + \lambda_1\lambda_2$$

When the PD controller is applied to the plant model, the resulting closed-loop characteristic equation can be written in terms of the unknown gains K_p and K_d . By matching the coefficients of this closed-loop polynomial with the desired polynomial above, one obtains two linear equations in K_p and K_d :

divide the entire equation by $(1-bK_d)$:

$$\frac{(1-bK_d)s^2}{1-bK_d} + \frac{(a+bK_p-cK_d)s}{1-bK_d} + \frac{cK_p}{1-bK_d}$$

$$s^2 + \frac{a + bK_p - cK_d}{1 - bK_d}s + \frac{cK_p}{1 - bK_d}$$

Comparing with

$$s^2 - (\lambda_1 + \lambda_2)s + \lambda_1\lambda_2$$

We get

$$cK_p - \lambda_1\lambda_2 bK_d = \lambda_1\lambda_2 \quad (7)$$

$$bK_p + (c - b(\lambda_1 + \lambda_2))K_d = a - (\lambda_1 + \lambda_2) \quad (8)$$

Here a , b , c come from the Transfer Function, and λ_1 , λ_2 are the chosen desired poles. These two equations (7) and (8) can be written compactly in matrix form

$$\begin{bmatrix} b & c \\ c - b(\lambda_1 + \lambda_2) & -b\lambda_1\lambda_2 \end{bmatrix} \begin{bmatrix} K_p \\ K_d \end{bmatrix} = \begin{bmatrix} a - (\lambda_1 + \lambda_2) \\ \lambda_1\lambda_2 \end{bmatrix}$$

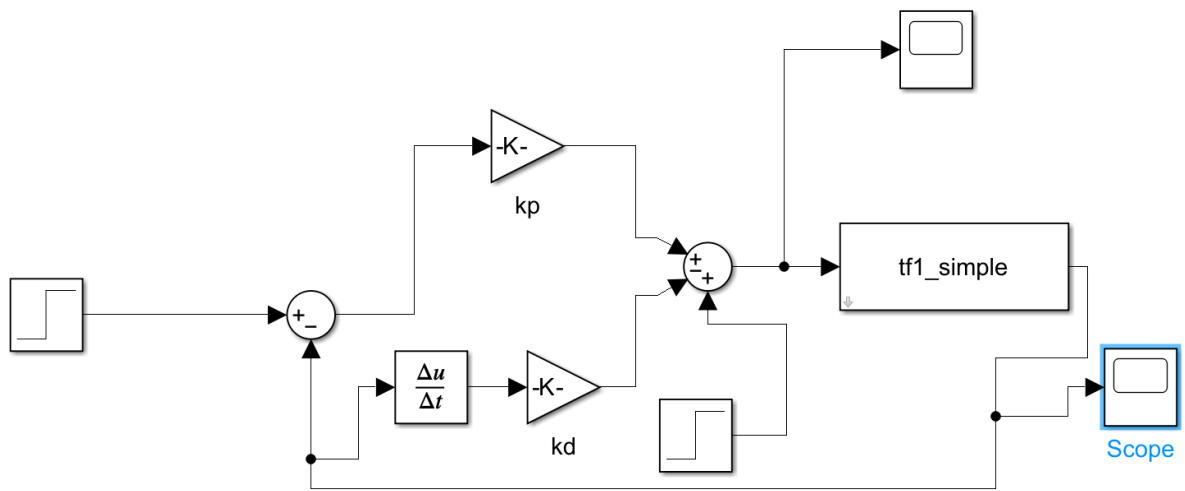


Fig 3.2.1: Block Diagram PD Controller

The Simulink block diagram in Fig. 3.2.1 shows this pure PD altitude controller connected to the plant block `tf1_simple`. The reference signal is compared with the measured altitude to generate the error, which then splits into two paths: the upper path multiplies the error by K_p , giving the proportional contribution, and the lower path differentiates the error (block $\Delta u/\Delta t$) and multiplies it by K_d , giving the derivative contribution. These two signals are summed and applied to the plant, whose output is fed back to form the closed loop and also sent to a scope. For this design, the target poles were chosen as $\lambda_1 = -2 + 0.4i$ and $\lambda_2 = -2 - 0.4i$; solving the pole-placement equations gives $K_p = 0.1772$ and $K_d = 0.0097$.

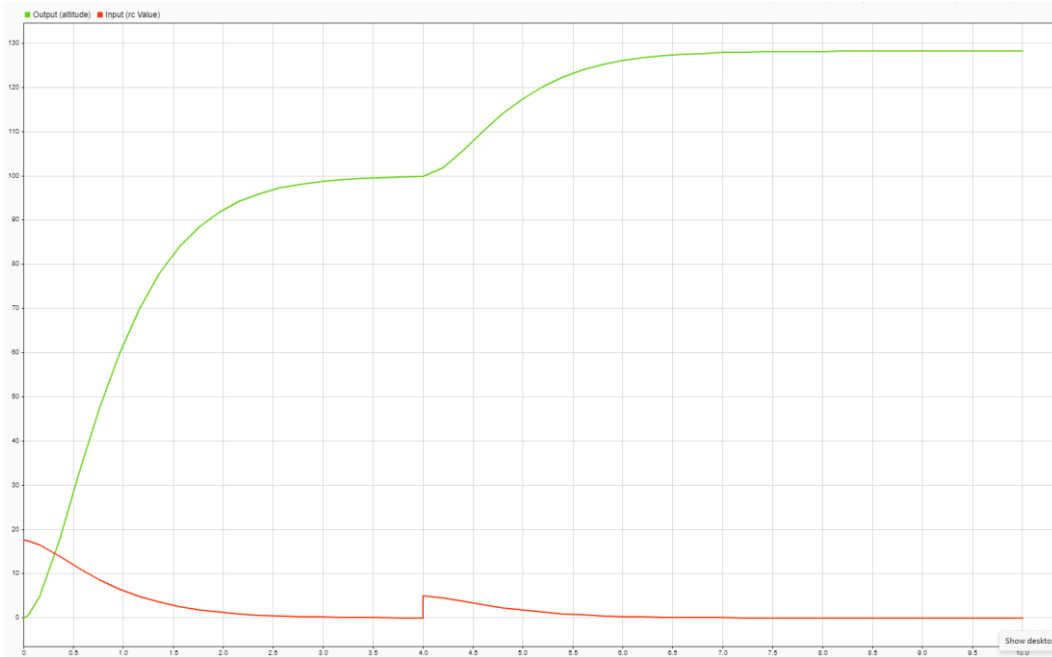


Fig 3.2.2: Simulation Result PD Controller

The output shown in Fig. 3.2.2 confirms that, with these gains, the system initially tracks the step command with a smooth rise and small overshoot, but when a step disturbance is applied at about 4 s, the response moves to a new level and does not return to the original reference. This behaviour highlights a key limitation of PD control although it stabilizes the system and shapes the transient response, it has no integral action, so it cannot remove steady-state error caused by constant disturbances or biases, and the altitude settles at a new offset instead of perfectly rejecting the disturbance.

3.2 I-PD Controller

I-PD controller, the idea is similar to the PD case, but now three desired closed-loop poles are chosen in the stable region of the s-plane, λ_1 , λ_2 , and λ_3 , so that the target characteristic polynomial is

$$(s-\lambda_1)(s-\lambda_2)(s-\lambda_3)=0$$

$$s^3 - (\lambda_1 + \lambda_2 + \lambda_3)s^2 + (\lambda_1\lambda_2 + \lambda_1\lambda_3 + \lambda_2\lambda_3)s - \lambda_1\lambda_2\lambda_3 = 0$$

The sums and products of the poles are usually grouped as

$$A_1 = \lambda_1 + \lambda_2 + \lambda_3,$$

$$A_2 = \lambda_1\lambda_2 + \lambda_1\lambda_3 + \lambda_2\lambda_3,$$

$$A_3 = \lambda_1\lambda_2\lambda_3,$$

$$P(s) = \frac{bs+c}{s^2+as} \quad C(s) = \frac{Ki}{s} - Kp - Kds$$

The closed-loop transfer function is: $G(S) = \frac{P(s)C(s)}{1+(P(s)C(s))}$

Substituting $P(s)$ and $C(s)$, $G(s) = \frac{(bs+c)\left(\frac{K_i}{s} - K_p - K_d s\right)}{s^2 + as + (bs+c)(K_i/s - K_p - K_d s)}$

$$G(s) = \frac{(bs+c)(K_i - K_p s - K_d s^2)}{s^3 + as^2 + (bs+c)(K_i - K_p s - K_d s^2)} \quad (9)$$

Expanding the denominator of Equation (9),

$$s^3 + as^2 + bK_i s + cK_i - bK_p s^2 - cK_p s - bK_d s^3 - cK_d s^2$$

Grouping terms,

$$(1 - bK_d)s^3 + (a - cK_d - bK_p)s^2 + (bK_i - cK_p)s + cK_i$$

Dividing by $(1-bK_d)$, $s^3 + \frac{a-cK_d-bK_p}{1-bK_d}s^2 + \frac{bK_i-cK_p}{1-bK_d}s + \frac{cK_i}{1-bK_d}$

The I-PD control law is

$$u = K_i \int e \, dt - K_p y - K_d \dot{y} \quad (10)$$

the integral gain K_i acts on the error, while the proportional and derivative parts act on the measured output

Matching coefficients,

$$\frac{a - cK_d - bK_p}{1 - bK_d} = -A_1$$

$$\frac{bK_i - cK_p}{1 - bK_d} = A_2$$

$$\frac{cK_i}{1 - bK_d} = -A_3$$

Multiplying by $(1-bK_d)$,

$$a - cK_d - bK_p = -A_1(1 - bK_d)$$

$$bK_i - cK_p = A_2(1 - bK_d)$$

$$cK_i = -A_3(1 - bK_d)$$

Rearranging,

$$-cK_i - bA_3K_d = A_3$$

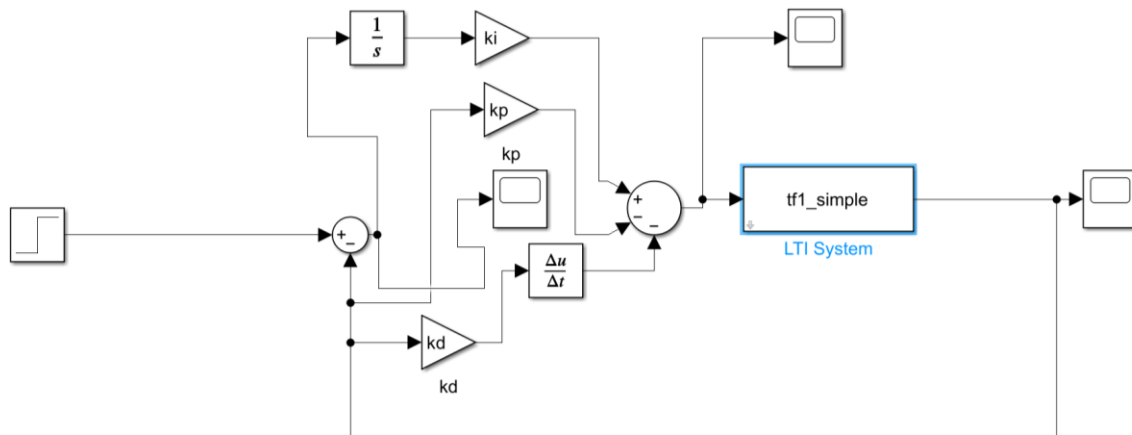
$$-cK_p - bK_i + bA_2K_d = -A_2$$

$$-((c + b) + bA_1)K_d = a + A_1$$

From these three equations can be written in matrix form with K_p , K_i , and K_d as unknowns.

For a given (a, b, c) and chosen poles $\lambda_1, \lambda_2, \lambda_3$

$$\begin{bmatrix} 0 & -c & -bA_3 \\ -c & -b & bA_2 \\ 0 & 0 & -(c + b) - bA_1 \end{bmatrix} \begin{bmatrix} K_p \\ K_i \\ K_d \end{bmatrix} = \begin{bmatrix} A_3 \\ -A_2 \\ a + A_1 \end{bmatrix}$$



Fig(3.2.1)

The Simulink diagram in Fig. 3.2.1 shows the I-PD controller connected to the Transfer Function tf1_simple without any time delay in the loop. On the left, the reference signal is compared with the measured output to form the error. This error is sent into an integrator block $1/s$; the integrated error is then multiplied by the gain K_i , which generates the integral term. This term slowly builds up whenever there is a steady error and drives the system so that the altitude eventually returns to the commanded value and the proportional and derivative. Because there is no time-delay block in this model, the behaviour represents the ideal I-PD actions use the measured output (feedback) instead of the error, which is the typical I-PD structure. The plant output is subtracted and scaled by K_p to form the proportional feedback, and its rate of change is computed by the $\Delta u/\Delta t$ block and scaled by K_d to form the derivative feedback. These three signals (integral from the error, proportional and derivative from the output(feedback)) are summed together and applied to the plant block tf1_simple. The plant output is fed back to close the loop and is also sent to scopes for plotting the altitude response. tcontroller acting directly on the linear plant model.

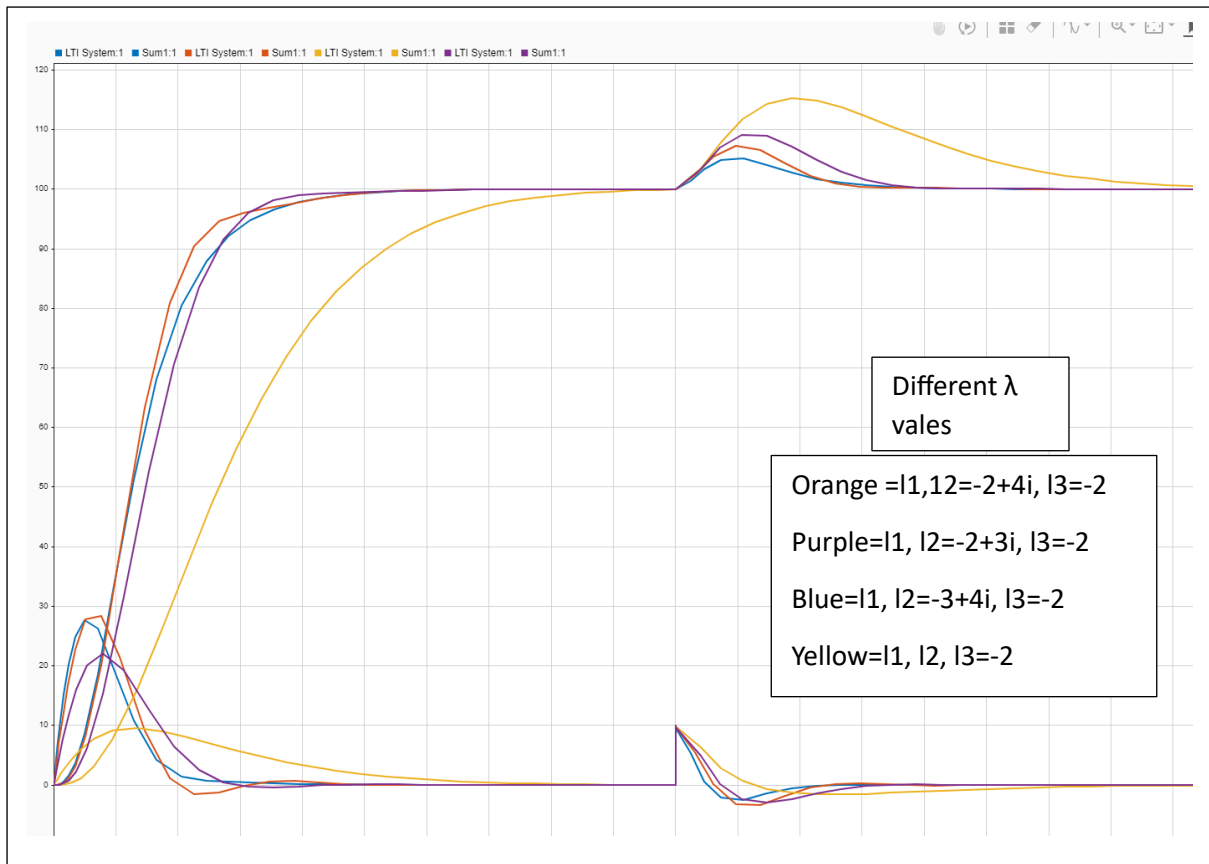


Fig 3.2.2: Simulation Result I-PD Controller

Figure 3.2.2 shows the altitude response of the I-PD controller in simulation, where the top plot is the drone altitude (output) and the bottom plot is the control input; the horizontal axis is time and the vertical axis in the top plot represents altitude. The different coloured curves correspond to different choices of desired closed-loop pole locations λ , so each colour is one set of $\lambda_1, \lambda_2, \lambda_3$ used in the I-PD design. At the start, all altitude curves rise smoothly to the commanded height without overshoot or oscillation, showing that the controller gives a clean and well-damped take-off. Around 7 seconds, a step disturbance is applied: the altitude briefly deviates from the reference and then returns to the required level. This recovery is mainly due to the integral part of the I-PD controller, which keeps integrating the error until the steady-state offset is removed and the drone comes back to the desired altitude.

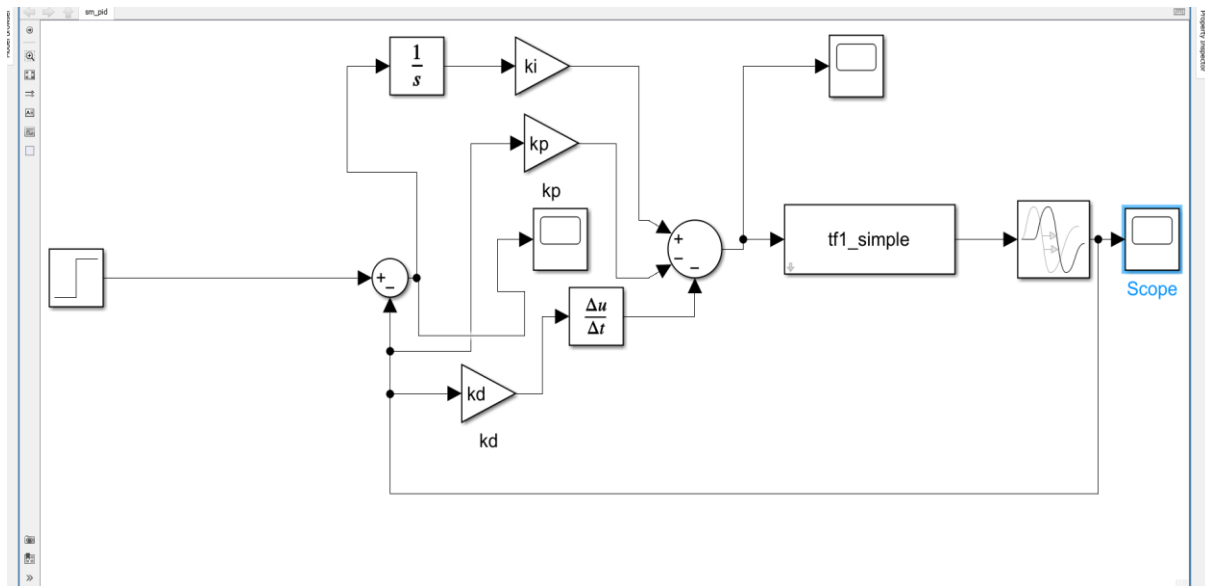


Fig 3.2.3: Simulink Block Diagram for PID with time delay

For the same I–PD controller, an additional time-delay block was inserted into the Simulink model, as shown in Fig. 3.2.3, to represent the effect of sensing and computation delays in a real drone altitude loop. With this delay included, the closed-loop response became poor: in the simulation plot of Fig. 3.2.4, the altitude no longer follows the command smoothly and the output looks sluggish and distorted instead of cleanly rising and settling. The drone's altitude drifts and shows large, slow deviations from the target, which means the controller can no longer keep the system stable and accurate once the delay is present.

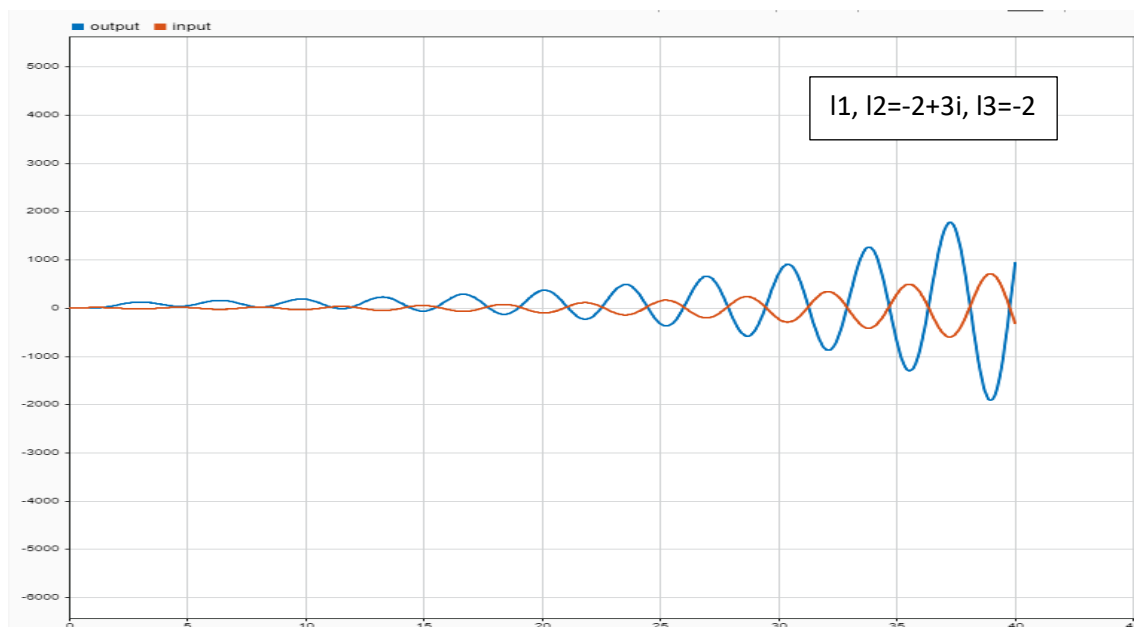
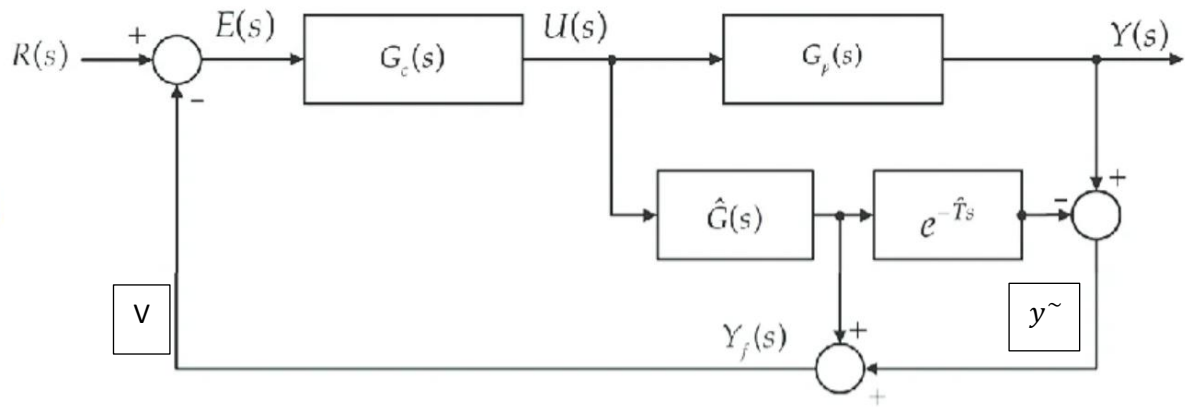


Fig 3.2.4: Simulation Result For PID with time delay

3.3 I-PD Controller with Time Delay and With Smith Predictor

The Smith predictor is specialised control strategy designed to handle system with significant time delay also known as dead time in many real world applications such as drone. There is a gap between a movement a control command is sent from the pc and the movement the drone actually or the control actually gets the result the standard feedback loop. These lag calls the controller to overreact to old data leading to aggressive Overshoot, constant oscillation and often total system instability. The fundamental goal of Smith predictor is to eliminate the delay from the feedback loop. It achieves this by using an internal mathematical model to predict the system's output instead of waiting for the real sensor data to arrive. The controller uses the model to see the estimated effect of its action instantly.



The core of the Smith Predictor is the construction of the "predicted" feedback signal (V). It combines the instantaneous model output ($\hat{y}$) with the residual error ($\tilde{y}$) to eliminate the effect of the delay from the loop.

$$V(s) = P_0(s)u(s) + \underbrace{[y(s) - P_0(s)e^{-sL}u(s)]}_{\tilde{y} \text{ (Residual)}}$$

When the model is accurate, the delayed terms cancel out, leaving:

$$V(s) = P_0(s)u(s)$$

So the controller becomes

$$U(s) = C(s)[R(s) - V(s)] \quad (11)$$

For I-PD Controller

$$C(s) = k_i \left(\frac{1}{s} \right) - k_p - k_d s \quad (12)$$

We get

$$U(s) = \left(Ki \left(\frac{1}{s} \right) - Kp - Kds \right) [R(s) - V(s)]$$

Figure (3.3 .1)shows a sibling model where an I-PD controller is combined with a Smith predictor to control the drones altitude the reference height is compared with the measured altitude to get an error signal these error goes into the integral block with the gain k_i while the measured altitude is fed back through a proportional and derivative gains k_p and k_d these three paths are added to form the control signal u This signal is then split One copy drives a model of the drone that includes the real time delay giving the actual output y , and the other copy drives The same model without time delay giving predicted output $\hat{y}$ This predictor uses this fast prediction together with The difference between $\hat{y}$ and the delayed output y to cancel out the effect of the delay. In simple terms, the controller does not have to wait for slow, delayed measurements instead it uses the model to see how the drone will move a little bit into the future, so it can react earlier and keep the altitude stable even when the real system has noticeable time delay.

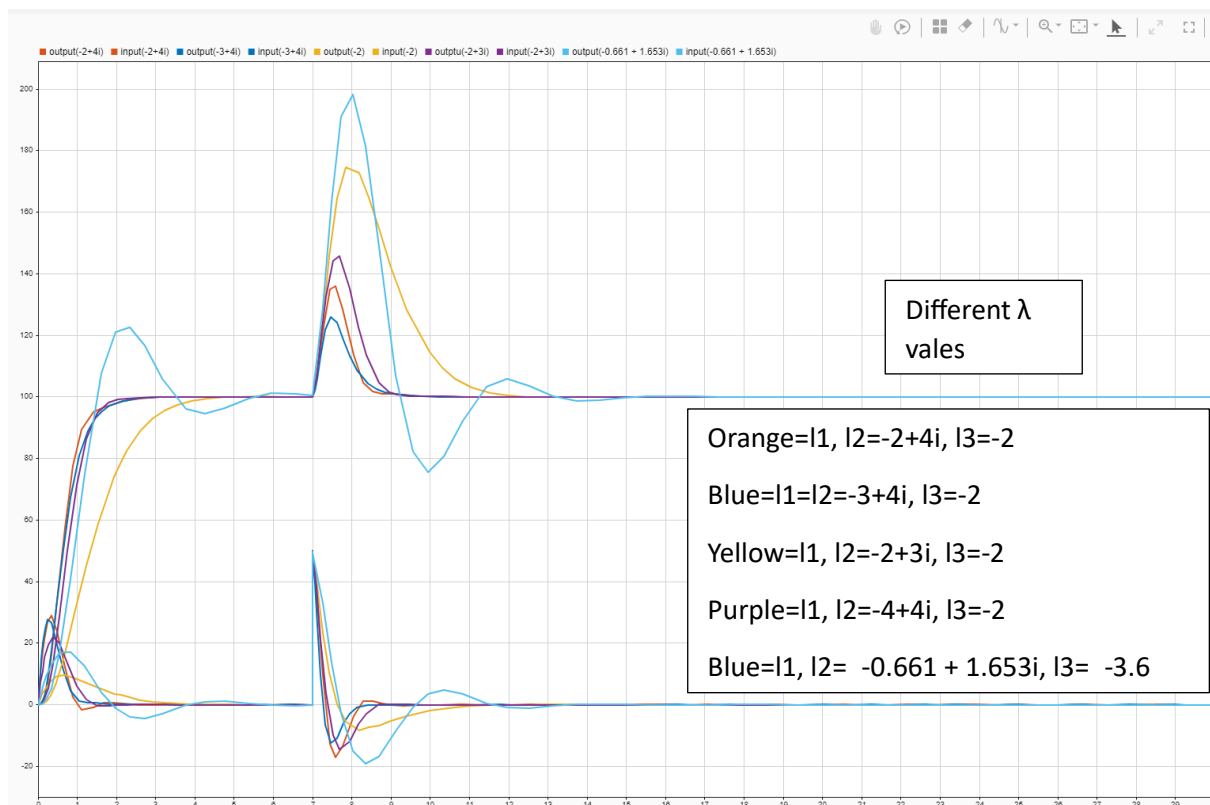


Fig 3.3.2

Figure 3.3.2 shows the altitude response of the I-PD controller with smith predictor (with time Delay) in simulation, where the top plot is the drone altitude (output) and the bottom plot is the control input; the horizontal axis is time and the vertical axis in the top plot represents altitude. The different coloured curves correspond to different choices of desired closed-loop pole locations λ , so each colour is one set of $\lambda_1, \lambda_2, \lambda_3$ used in the I-PD design. At the start, all

altitude curves rise smoothly to the commanded height without overshoot or oscillation, showing that the controller gives a clean and well-damped take-off. Around 7 seconds, a step disturbance is applied: the altitude briefly deviates from the reference and then returns to the required level. This graph shows that the Smith predictor successfully compensates for the delay and keeps the drone stable.

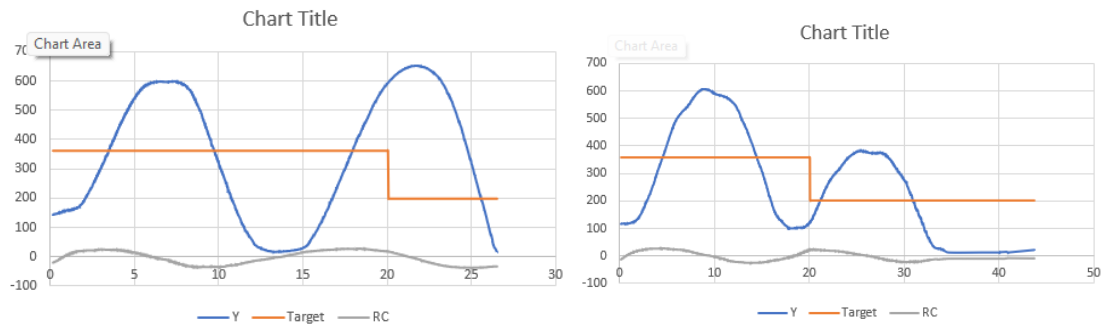
4. Experiment Results

4.1 Experiment setup

For the experiment, the drone was placed about 9.3 m away from the ball, the same distance used earlier for system identification. A PC running Python code was connected to the drone over Wi-Fi and received the live video stream from the onboard camera. YOLOv8 was used to detect the ball in each frame and report its horizontal image position in pixels. At the start of the test, the code treated a ball centre position is 360 pixels and move drone of about 600 pixels as the “correct” location in the image. After 20 seconds, this desired position was changed to 100 pixels, creating a step change in the visual target that forces the drone to move and re-centre the ball. The pixel error (current ball position minus desired pixel position) in this experiment two versions of the control program were run: one using a basic PID controller and another using a PID plus Smith predictor to compensate for the delay in the Wi-Fi and processing loop.

4.2 Experiment using I-PD controller

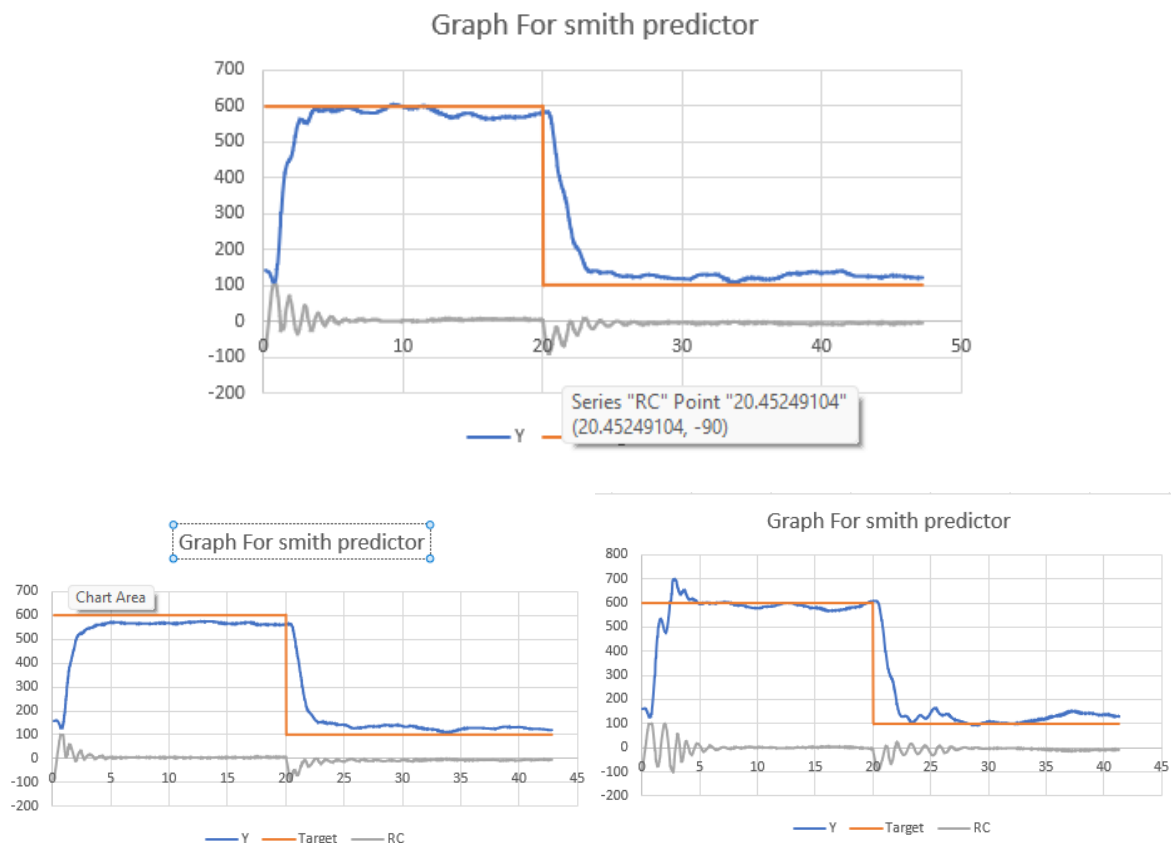
In the first experiment, the Python code used only a basic PID controller. When the key “t” was pressed on the keyboard, the command `tello.takeoff()` was sent and the drone took off. The drone then flew upward and stopped when YOLO detected the sports ball and reported that its image position was at about 360 pixels. After 20 seconds, the desired position was changed to 200 pixels so that the drone had to move again to re-centre the ball. In this pure PID case the drone motion was very oscillatory just after take-off, and only later did the oscillations become smaller, as can be seen in (Fig. 4.2.1) to reduce noise and sudden jumps in the control signal, a simple low-pass filter with parameter α was also used on the measured signal



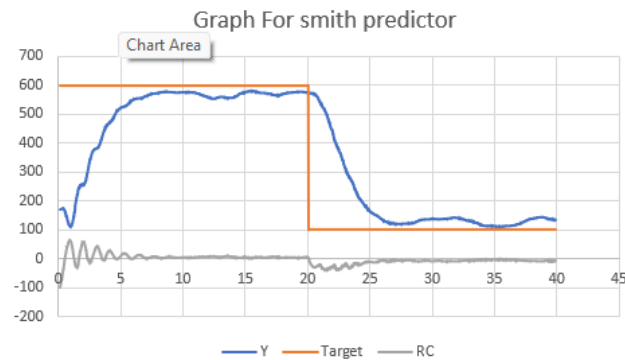
With $k_p = 0.14$, $k_i = 0.04$ and $k_d = 0.10$ and $\lambda_1 = -0.28 - 0.3i$, $\lambda_2 = -0.28 + 0.3i$, $\lambda_3 = -5.3$

4.3 Experiment using I-PD controller with smith predictor

In the second experiment, the Python code used a PID controller together with a Smith predictor to deal with the delay in the drone-camera-Wi-Fi loop. The Smith predictor class keeps a simple internal model of how the drone responds to the control signal and uses this model to predict what the altitude should be right now, without delay. At every time step it compares this “instant” model output with an older, delayed copy stored in a small history buffer, and adds the difference to the real sensor measurement. In practice, this means the PID controller receives a corrected signal that behaves as if there were almost no delay, so the drone moves much more smoothly and stays closer to the desired ball position than in the basic PID experiment.



For $k_p:0.343$, $k_i:0.5$, $k_d:0.056$ for of $\lambda_1 = -0.661 + 1.653i$; $\lambda_2 = -0.661 + 1.653i$; $\lambda_3 = -3.6$



4.Conclusion

This project focused on designing and implementation of an autonomous altitude control system for the Ryze Tello drone using two approaches a conventional PID controller and PID controller combined with a smith predictor the goal was to understand how system delays caused by WIFI commutation and YOLO-based vision processing affect performance and how they can be effectively managed From both simulation and real world experiments the PID controller was able to stabilize the drone and maintain its altitude under normal conditions. However, when delays were present, its performance degraded. The drone showed noticeable oscillations, slower settling time, and less accurate tracking, especially during quick altitude changes. These issues highlighted the limitations of using a standard PID controller in delay-sensitive systems. To address this, a Smith Predictor was integrated with the PID controller. By using a model of the system to predict future behaviour, the combined approach effectively compensated for the delay. This resulted in smoother responses, reduced oscillations, lower overshoot, and faster settling times. Overall, the drone maintained a more stable and accurate altitude during autonomous flight. In conclusion, while a PID controller alone can provide basic control, it is not sufficient for systems with significant delays. The addition of a Smith Predictor greatly improves performance, making the control system more reliable and better suited for real-time, vision-based drone applications where delays are unavoidable.